

Aula 11: Arquivos de texto

Manipulação e persistência de dados

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores, trabalhamos com o objetos iteráveis no Python e subprogramação
 - * Discutimos listas, tuplas, conjuntos e dicionários
 - * Aprendemos a diferenciar e construir funções iterativas e recursivas
 - * Criamos programas e módulos
- ⌋ Até o presente momento, toda manipulação de dados que fizemos foi em memória principal (memória volátil)
- ⌋ Nesta aula, vamos explorar brevemente o conceito de persistência de dados e vamos aprender a lidar com um dos tipos de arquivos mais simples de se tratar, os **arquivos de texto**

Persistência de dados

- ⌋ Até o momento, todos os programas que fizemos na disciplina consideravam a saída de dados padrão, i.e., o terminal exibido na tela do computador
- ⌋ Nossos programas, quando executados, tem seus dados e instruções armazenados na **memória principal** do computador.
 - * Memória de alta velocidade de acesso
 - * Capacidade de armazenamento mediana
 - * Volátil (os dados só existem enquanto houver energia)
- ⌋ Em outras palavras, nossos programas estavam sujeitos à disponibilidade de energia e de dados manualmente inseridos pelo usuário para funcionar

Persistência de dados

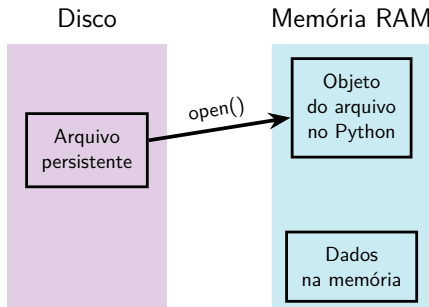
- ⌋ Apesar de executarem em memória principal, a grande maioria das informações (dados e programas) de um computador estão armazenadas no que chamamos de **memória secundária**
 - * Memória secundária é memória não-volátil
 - * Principalmente representada por HD's, SSD's, pen-drives
 - * Proveem um meio de armazenamento persistente de dados
- ⌋ Imagine que você tenha um grande cadastro de usuários de um site comercial qualquer
 - * Se essas informações não estiverem registradas em um arquivo, seria necessário inserir esses dados sempre que o computador (servidor) fosse desligado ou sofresse um desligamento forçado
 - * Ou seja, não haveria armazenamento de dados, apenas processamento deles

- ☾ Um **arquivo** é uma área de memória secundária onde gravamos e de onde lemos dados
- ☾ Podemos pensar em dois tipos de arquivos
 - * **Arquivos de textos:** são uma sequência estruturada de linhas com sequência de caracteres que pode ser lida por qualquer editor de texto
 - * **Arquivos binários:** é qualquer arquivo que não seja arquivo de texto padrão, como pdf, doc, imagens e etc
- ☾ Nos arquivos binários, os dados são armazenados de uma forma diferente, de modo que apenas programas específicos conseguem decodificar sua organização

- ☾ Os arquivos com extensão .txt (conhecidos como arquivos de texto simples ou *plain text*) são uma das formas mais básicas e universais de armazenar dados num computador
 - * Não guardam estilos visuais — como negrito, itálico, cores, fontes ou tamanhos de letra
 - * Por não conterem metadados complexos ou imagens, costumam ocupar apenas alguns kilobytes (KB)
 - * Qualquer sistema operacional e praticamente qualquer linguagem de programação consegue ler e escrever em arquivos .txt sem precisar de softwares adicionais
 - * Como o arquivo armazena apenas sequências numéricas (bytes), ele depende de uma tabela de codificação para interpretar os caracteres: UTF-8, ASCII ou ISO-8859-1

Ciclo de manipulação de um arquivo

- ☾ A função **open()** é a porta de entrada para qualquer manipulação de arquivos em Python
 - * Ela funciona solicitando ao Sistema Operacional a criação de um ponteiro de arquivo (*file descriptor* ou *stream*), permitindo que seu programa leia ou escreva dados do disco para a memória RAM



- ⌋ Podemos abrir o arquivo usando dois comandos diferentes

```
<var> = open(<arq>,<modo>,<codif>)
```

```
with open(<arq>,<modo>,<codif>) as <var>:
```

- * <var> é o nome da variável do objeto do arquivo
- * <arq> é o nome do arquivo com a extensão .txt (*string*)
- * <codif> é o tipo de codificação do texto (*string*), prefira usar "utf-8"
- * <modo> é a *string* que denota o tipo de operação a ser executada no arquivo

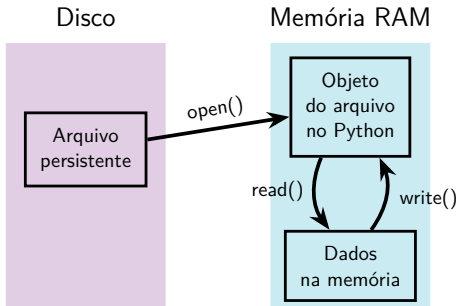
- ⌋ <modo> é a *string* que denota o tipo de operação a ser executada no arquivo
- * b : indica que o arquivo é lido como binário (não-texto), é usado em tratamento de dados como imagens e outros
 - * w : indica escrita, apaga o conteúdo de um arquivo existente ou cria um novo arquivo
 - * r : indica leitura (é o valor padrão deste parâmetro), exige que o arquivo exista (gera um erro se o arquivo não existir)
 - * a : indica escrita a partir do final do arquivo (*append*)
 - * + : indica uma atualização, sempre usado em conjunto com r, w ou a para permitir leitura e escrita de um arquivo

Abrindo o arquivo

- ⌋ O método **open()** de forma direta, como em `file = open(...)`, deixa o arquivo aberto na memória até que você chame explicitamente o método **.close()**
 - * Se o seu código falhar ou der erro antes da linha do **.close()**, o arquivo ficará preso na RAM e travado no sistema
- ⌋ Para contornar este tipo de problema, usamos a palavra reservada **with**, que cria o que chamamos **gerenciador de contexto**
 - * O **gerenciador de contexto** é um recurso que permite gerenciar recursos de forma segura, mesmo em caso de erro
 - * O Python garantirá o fechamento do arquivo neste caso, independente de uma falha ocorrer ou não
 - * Com o gerenciador de contexto, não precisamos usar o **.close()** explicitamente

Ciclo de manipulação de um arquivo

- As operações de leitura e escrita que fizermos serão todas entre as variáveis do nosso programa e o objeto do nosso arquivo (*buffer* de memória)
- * Tudo aqui ocorre em memória principal (a memória RAM)



Leitura de dados

- ☾ A leitura de dados para variáveis que iremos manipular pode ocorrer das seguintes formas
 - * Considere que foi criado um *buffer* de memória chamado **arquivo** com a execução de `with open(...)` as `arquivo` :
 - * **arquivo.read()** carrega todo o conteúdo do arquivo de uma só vez para uma única *string*, de modo que as linhas estão separadas pelo caracter `\n`
 - * Podemos ler uma linha por vez com **arquivo.readline()** ou fazendo uma iteração do tipo **for linha in arquivo** (lembre-se que o caracter `\n` aparecerá ao final de cada linha)
 - * **arquivo.readlines()** lê o arquivo inteiro e retorna uma lista de *strings*, onde cada elemento da lista corresponde a uma linha
- ☾ Carregar o arquivo todo de uma vez pode ser super inconveniente caso o arquivo seja muito grande

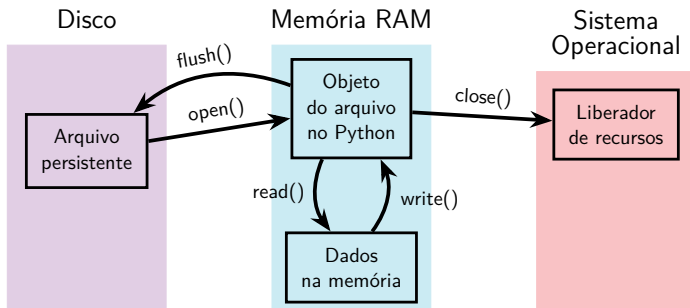
Quebra de linha

- ☾ Arquivos gerados no Windows possuem uma peculiaridade se comparados com arquivos gerados no Linux ou macOS, por exemplo
 - * No Linux/macOS, o padrão de indicação de quebra de linha é o caracter `\n`
 - * No Windows, o padrão de indicação de quebra de linha é a combinação de caracteres `\r\n`
 - * O Python lida com isso de forma automática ao abrir arquivos em **modo texto**
 - * Ele converte o padrão do sistema (como `\r\n` no Windows) para `\n` ao ler, e volta para `\r\n` ao escrever
 - * Se você abrir um arquivo em modo binário, o Python **desativa** toda a conversão automática de quebras de linha e codificação de texto

- ☾ Para a escrita de dados, temos as seguintes opções
 - * **arquivo.write(linha)** escreve a *string* **linha** no *buffer*, sendo necessário incluir o caracter `\n` manualmente para pular linha no arquivo texto
 - * **arquivo.writelines(linhas)** escreve as *strings* da lista **linhas** no *buffer*, sendo necessário incluir o caracter `\n` manualmente para cada uma das linhas sendo escritas
 - * Podemos ainda usar o tradicional comando **print(linha,file=arquivo)** para adicionar a *string* **linha** ao *buffer* passado como argumento nomeado **file** (aqui, o caracter `\n` é inserido automaticamente pelo comando)
- ☾ Os comandos de escrita só podem escrever *strings*, então tipos de dados diferentes devem ser devidamente convertidos para *string* antes de escrever

Ciclo de manipulação de um arquivo

- ⌋ A persistência dos dados (envio das informações do *buffer* para o disco) pode ser feita de duas formas
 - * **.flush()** força a atualização do arquivo sem fechar a conexão do *buffer*
 - * **.close()** executa o *flush* internamente e fecha a conexão



Fechando o arquivo

- ⌋ O método **close()**, como dito anteriormente, serve não só para espelhar as modificações do *buffer* no disco, como também encerra a conexão com o arquivo
 - * Em outras palavras, a região de memória é liberada, assim como os travamentos no arquivo sendo tratado
- ⌋ O gerenciador de contexto de contexto executa o fechamento do arquivo automaticamente, mesmo diante da existência de um erro
 - * Logo, todas as alterações existentes no *buffer* até o momento do fechamento serão executadas
- ⌋ Os dados não serão salvos apenas nos seguintes cenários: queda de energia, desligamento forçado, **kernel panic**, o SO encerrar o Python à força e execução de **os._exit()**

Encerramento da disciplina

- ☾ Ao longo deste período, discutimos uma série de informações básicas sobre como programar em Python
 - * Nem sempre vimos a forma mais adequada de fazer algo em Python pelo simples fato de este curso não abordar, por exemplo, os conceitos de orientação a objetos
- ☾ O objetivo principal desta disciplina é introduzir os alunos ao universo da programação usando a Linguagem Python como objeto de estudo
 - * Lembrem-se que a maior dificuldade de se resolver um problema não está em saber a sintaxe de uma linguagem de programação
 - * O núcleo de todo programa é o **algoritmo** desenvolvido
 - * Portanto, pensem sempre com muita calma na forma como vocês pretendem atacar um problema

